封装服务器发送队列

Posted on 2023-04-22 19:58:22 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2OmTLf4Ahbksklnru69zfSz6rg2)

简介

前文介绍了通过智能指针实现伪闭包的方式延长了session的生命周期，而实际使用的服务器并不是应答式，而是全双工通信方式，服务器一直监听写事件，接收对端数据，可随时发送数据给对端，今天介绍如何封装异步的发送接口，因为多次发送时，异步的发送要保证回调触发后再次发送才能确保数据是有序的，这一点我们已经在前文异步发送函数介绍的时候提到了。

Server和Session分离

将Server修改为CServer并分离到CServer.h中，然后将Session修改为CSession分离到CSession.h中。
CSession.h中类的声明如下，和之前的Session内容一样，就是修改了类名，放在CSession.h中

```
#include <iostream>
#include <boost/asio.hpp>
#include <map>
#include <boost/uuid/uuid_generators.hpp>
#include <boost/uuid/uuid_io.hpp>
using boost::asio::ip::tcp;
using namespace std;
class CServer;
class CSession :public std::enable_shared_from_this<CSession>
{
public:
    CSession(boost::asio::io_context& ioc, CServer* server) :_socket(ioc), _server(server) {
        boost::uuids::uuid  a_uuid = boost::uuids::random_generator()();
        _uuid = boost::uuids::to_string(a_uuid);
    }
    tcp::socket& Socket() {
        return _socket;
    }

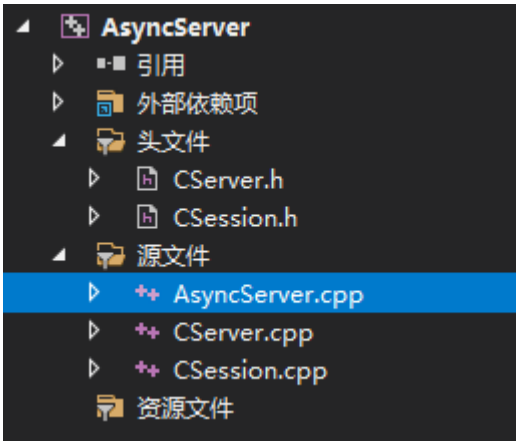
    ~CSession() {
        std::cout << "session destruct delete this " << this << endl;
    }

    void Start();
    std::string& GetUuid();
private:
    void handle_read(const boost::system::error_code& error,
        size_t bytes_transferred, shared_ptr<CSession> _self_shared);
    void handle_write(const boost::system::error_code& error, shared_ptr<CSession> _self_sha
red);
    tcp::socket _socket;
    enum { max_length = 1024 };
    char _data[max_length];
    CServer* _server;
    std::string _uuid;
};
```

CServer.h中声明如下，内容前文没变化，就是将Server内容写入CServer.h中

```cpp
#include <boost/asio.hpp>
#include "CSession.h"
#include <memory.h>
#include <map>
using namespace std;
using boost::asio::ip::tcp;
class CServer
{
public:
    CServer(boost::asio::io_context& io_context, short port);
    void ClearSession(std::string);
private:
    void HandleAccept(shared_ptr<CSession>, const boost::system::error_code & error);
    void StartAccept();
    boost::asio::io_context &_io_context;
    short _port;
    tcp::acceptor _acceptor;
    std::map<std::string, shared_ptr<CSession>> _sessions;
};
```

整体目录变为



数据节点设计

我们设计一个数据节点MsgNode用来存储数据

```
class MsgNode
{
    friend class CSession;
public:
    MsgNode(char * msg, int max_len) {
        _data = new char[max_len];
        memcpy(_data, msg, max_len);
    }

    ~MsgNode() {
        delete[] _data;
    }

private:
    int _cur_len;
    int _max_len;
    char* _data;
};
```

1 _cur_len表示数据当前已处理的长度(已经发送的数据或者已经接收的数据长度)，因为一个数据包存在未发送完或者未接收完的情况。
2 _max_len表示数据的总长度。
3 _data表示数据域，已接收或者已发送的数据都放在此空间内。

封装发送接口

首先在CSession类里新增一个队列存储要发送的数据，因为我们不能保证每次调用发送接口的时候上一次数据已经发送完，就要把要发送的数据放入队列中，通过回调函数不断地发送。而且我们不能保证发送的接口和回调函数的接口在一个线程，所以要增加一个锁保证发送队列安全性。
同时我们新增一个发送接口Send

```
void Send(char* msg,  int max_length);
std::queue<shared_ptr<MsgNode> > _send_que;
std::mutex _send_lock;
```

实现发送接口

```
void CSession::Send(char* msg, int max_length) {
    bool pending = false;
    std::lock_guard<std::mutex> lock(_send_lock);
    if (_send_que.size() > 0) {
        pending = true;
    }
    _send_que.push(make_shared<MsgNode>(msg, max_length));
    if (pending) {
        return;
    }

    boost::asio::async_write(_socket, boost::asio::buffer(msg, max_length),
        std::bind(&CSession::HandleWrite, this, std::placeholders::_1, shared_from_this()));
}
```

发送接口里判断发送队列是否为空，如果不为空说明有数据未发送完，需要将数据放入队列，然后返回。如果发送队列为空，则说明当前没有未发送完的数据，将要发送的数据放入队列并调用async_write函数发送数据。
回调函数实现

```
void CSession::HandleWrite(const boost::system::error_code& error, shared_ptr<CSession> _
self_shared) {
    if (!error) {
        std::lock_guard<std::mutex> lock(_send_lock);
        _send_que.pop();
        if (!_send_que.empty()) {
            auto &msgnode = _send_que.front();
            boost::asio::async_write(_socket, boost::asio::buffer(msgnode->_data, msgnode->_ma
x_len),
                std::bind(&CSession::HandleWrite, this, std::placeholders::_1, _self_shared));
        }
    }
    else {
        std::cout << "handle write failed, error is " << error.what() << endl;
        _server->ClearSession(_uuid);
    }
}
```

判断发送队列是否为空，为空则发送完，否则不断取出队列数据调用async_write发送，直到队列为空。

修改读回调

因为我们要一直监听对端发送的数据，所以要在每次收到数据后继续绑定监听事件

```
1. void CSession::HandleRead(const boost::system::error_code& error, size_t  bytes_transferred, shared_ptr<CSession> _self_shared){
2.     if (!error) {
3.         cout << "read data is " << _data << endl;
4.         //发送数据
5.         Send(_data, bytes_transferred);
6.         memset(_data, 0, MAX_LENGTH);
7.         _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH), std::bind(&CSession::HandleRead, this,
8.             std::placeholders::_1, std::placeholders::_2, _self_shared));
9.     }
10.    else {
11.        std::cout << "handle read failed, error is " << error.what() << endl;
12.        _server->ClearSession(_uuid);
13.    }
14. }
```

总结

该服务器虽然实现了全双工通信，但是仍存在缺陷，比如粘包问题未处理，下一版本实现粘包处理。
源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2956) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(518) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)